Name :- Akshat Chauhan

CU24250136

BTech CSE 3rd Year SECTION - "C"

Roll No. :-05

Experiment No. 5

Experiment Name

Feature Extraction and Image Analysis using SIFT and HOG Descriptors

Aim:

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, image matching and computer vision applications.



Step 1: Import Required Libraries

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure

# Removed tkinter as it causes issues in Colab's non-GUI environment
# from tkinter import Tk, filedialog
```

Step 2: Select First Image from Computer

```
from google.colab import files
import io

print("Please select the first image file.")
uploaded_files = files.upload()

if uploaded_files:
    image_path_1 = list(uploaded_files.keys())[0]
    print(f"First image selected: {image_path_1}")
else:
    print("No first image selected.")
    exit()
```

```
Please select the first image file.
Choose Files No file chosen  Uploadwidgetisonlyavailablewhenthecellhasbeen executed in the current browser session. Please rerun this cell to enable.
Saving Screenshot 2025-02-25 135945.png to Screenshot 2025-02-25 135945.png
First image selected: Screenshot 2025-02-25 135945.png
```

Step 3: Select Second Similar Image from Computer

```
from google.colab import files
import io

print("Please select the Second image file.")
uploaded_files = files.upload()

if uploaded_files:
    image_path_2 = list(uploaded_files.keys())[0] # Corrected variable assignment
    print(f"Second image selected: {image_path_2}")
else:
    print("No Second image selected.")
    exit()
```

```
Please select the Second image file.
Choose Files No file chosen  Uploadwidgetisonlyavailablewhenthecellhasbeenexecuted in the current browser session. Please rerun this cell to enable.
Saving Screenshot 2025-02-20 150759.png to Screenshot 2025-02-20 150759 (1).png
Second image selected: Screenshot 2025-02-20 150759 (1).png
```

Step 4: Load Images

```
image1 = cv2.imread(image_path_1)
image2 = cv2.imread(image_path_2)

if image1 is None:
    print("Error: Unable to load first image.")
    exit()

if image2 is None:
    print("Error: Unable to load second image.")
    exit()

print("Both images loaded successfully.")

print("First Image:", image_path_1)
print("Second Image:", image_path_2)
```

```
Both images loaded successfully.
First Image: Screenshot 2025-02-25 135945.png
Second Image: Screenshot 2025-02-20 150759 (1).png
```

Step 5: Convert Images to Grayscale

```
gray1 = cv2.cvtColor(
    image1,
    cv2.COLOR_BGR2GRAY
)

gray2 = cv2.cvtColor(
    image2,
    cv2.COLOR_BGR2GRAY
)

print("\n========== IMAGE PREPROCESSING ==========")

print("First image converted to grayscale.")
print("Second image converted to grayscale.")
```

```
========== IMAGE PREPROCESSING ==========
First image converted to grayscale.
Second image converted to grayscale.
```

Step 6: Display Original Images

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.imshow(
    cv2.cvtColor(image1, cv2.COLOR_BGR2RGB)
)

plt.title("First Image")
plt.axis("off")


plt.subplot(1, 2, 2)

plt.imshow(
    cv2.cvtColor(image2, cv2.COLOR_BGR2RGB)
)

plt.title("Second Image")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Step 7: Create SIFT Detector

```
sift = cv2.SIFT_create()
```

Step 8: Detect SIFT Keypoints and Descriptors

```
keypoints1, descriptors1 = sift.detectAndCompute(
    gray1,
    None
)

keypoints2, descriptors2 = sift.detectAndCompute(
    gray2,
    None
)

print("\n========== SIFT FEATURE EXTRACTION ==========")

print(
    "Number of SIFT Keypoints in Image 1:",
    len(keypoints1)
)

print(
    "Number of SIFT Keypoints in Image 2:",
    len(keypoints2)
)

if descriptors1 is not None:
    print(
        "SIFT Descriptor Shape - Image 1:",
        descriptors1.shape
    )
```

```
if descriptors2 is not None:
    print(
        "SIFT Descriptor Shape - Image 2:",
        descriptors2.shape
    )
```

```
========== SIFT FEATURE EXTRACTION ==========
Number of SIFT Keypoints in Image 1: 191
Number of SIFT Keypoints in Image 2: 1227
SIFT Descriptor Shape - Image 1: (191, 128)
SIFT Descriptor Shape - Image 2: (1227, 128)
```

Step 9: Draw SIFT Keypoints

```
sift_image1 = cv2.drawKeypoints(
    image1,
    keypoints1,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

sift_image2 = cv2.drawKeypoints(
    image2,
    keypoints2,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
```

Step 10: Display SIFT Keypoints

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.imshow(
    cv2.cvtColor(
        sift_image1,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("SIFT Keypoints - Image 1")
plt.axis("off")


plt.subplot(1, 2, 2)

plt.imshow(
    cv2.cvtColor(
        sift_image2,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("SIFT Keypoints - Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()

print("SIFT keypoints visualized successfully.")
```




```
SIFT keypoints visualized successfully.
```

Step 11: HOG Feature Extraction - Image 1

```
gray1_float = gray1 / 255.0

hog_features1, hog_image1 = hog(
    gray1_float,
    orientations=9,
    pixels_per_cell=(8,  8),
    cells_per_block=(2,  2),
    block_norm="L2-Hys",
    visualize=True
)
```

Step 12: HOG Feature Extraction - Image 2

```
gray2_float = gray2 / 255.0

hog_features2, hog_image2 = hog(
    gray2_float,
    orientations=9,
```

```
    pixels_per_cell=(8,  8),
    cells_per_block=(2,  2),
    block_norm="L2-Hys",
    visualize=True
)
```

Step 13: Print HOG Information

```
print("\n========== HOG FEATURE EXTRACTION ==========")

print(
    "HOG Feature Length - Image 1:",
    len(hog_features1)
)

print(
    "HOG Feature Length - Image 2:",
    len(hog_features2)
)

print("Orientations:",
print("Pixels per Cell:", (8,
8)) print("Cells per Block:", (2,
2))
```

```
========== HOG FEATURE EXTRACTION ==========
HOG Feature Length - Image 1: 1139544
HOG Feature Length - Image 2: 1139544
Orientations: 9
Pixels per Cell: (8, 8)
Cells per Block: (2, 2)
```

Step 14: Improve HOG Visualization

```
hog_image1_rescaled = exposure.rescale_intensity(
    hog_image1,
    in_range=(0, 10)
)

hog_image2_rescaled = exposure.rescale_intensity(
    hog_image2,
    in_range=(0, 10)
)
```

Step 15: Display HOG Images

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.imshow(
    hog_image1_rescaled,
    cmap="gray"
)

plt.title("HOG Visualization - Image 1")
plt.axis("off")

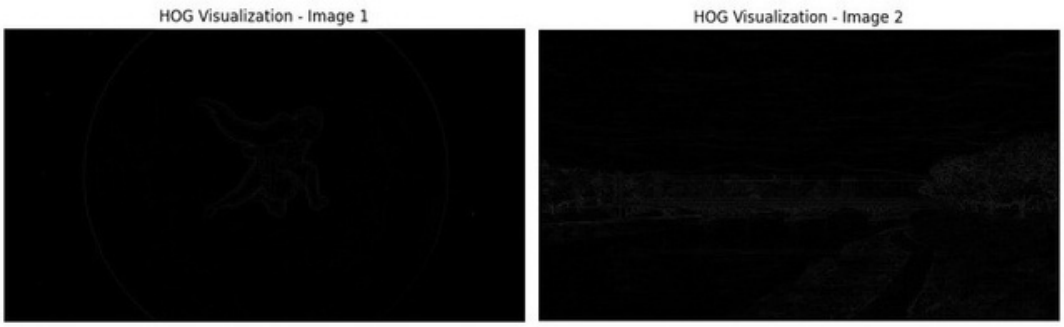

plt.subplot(1, 2, 2)

plt.imshow(
    hog_image2_rescaled,
    cmap="gray"
)

plt.title("HOG Visualization - Image 2")
plt.axis("off")

plt.tight_layout()
plt.show()

print("HOG descriptors visualized successfully.")
```




```
HOG descriptors visualized successfully.
```

Step 16: SIFT Image Matching

```
print("\n========== SIFT IMAGE MATCHING ==========")

if descriptors1 is not None and descriptors2 is not None:

    bf = cv2.BFMatcher()

    matches = bf.knnMatch(
```

```
        descriptors1,
        descriptors2,
        k=2
    )

    good_matches = []

    for match_pair in matches:

        if len(match_pair) == 2:

            m, n = match_pair

            if m.distance < 0.75 * n.distance:
                good_matches.append(m)

    print(
        "Total SIFT Matches:",
        len(matches)
    )

    print(
        "Good SIFT Matches:",
        len(good_matches)
    )
```

```
========== SIFT IMAGE MATCHING ==========
Total SIFT Matches: 191
Good SIFT Matches: 1
```

Step 17: Display SIFT Matches

```
if descriptors1 is not None and descriptors2 is not None:

    match_image = cv2.drawMatches(
        image1,
        keypoints1,
        image2,
        keypoints2,
        good_matches,
        None,
        flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
    )

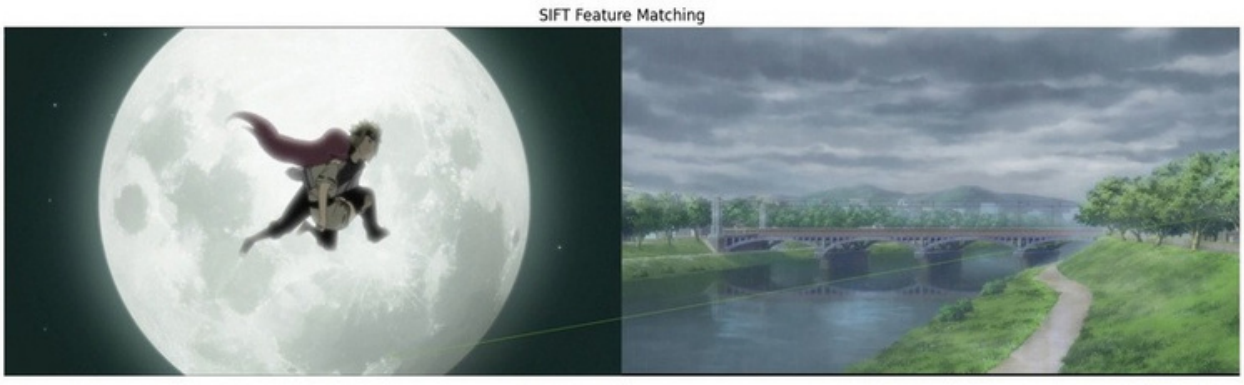
    plt.figure(figsize=(14, 7))

    plt.imshow(
        cv2.cvtColor(
            match_image,
            cv2.COLOR_BGR2RGB
        )
    )

    plt.title("SIFT Feature Matching")
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    print("SIFT feature matching visualization completed.")
```



Step 18: Compare HOG Features

```
if len(hog_features1) == len(hog_features2):

    hog_distance = np.linalg.norm(
        hog_features1 - hog_features2
    )

    print(
        "\nHOG Feature Euclidean Distance:",
        round(hog_distance, 4)
    )

else:

    print(
        "\nHOG feature lengths are different."
    )

    print(
        "Resize both images to the same dimensions "
```

```
        "for direct HOG comparison."
    )
```

```
HOG Feature Euclidean Distance: 145.8479
```

Step 19: Resize Images for Equal Dimensions

```
common_size = (256, 256)

resized_gray1 = cv2.resize(
    gray1,
    common_size
)

resized_gray2 = cv2.resize(
    gray2,
    common_size
)
```

Step 20: Calculate HOG Features on Equal-Sized Images

```
resized_hog1 = hog(
    resized_gray1  /  255.0,
    orientations=9,
    pixels_per_cell=(8,  8),
    cells_per_block=(2,  2),
    block_norm="L2-Hys"
)

resized_hog2 = hog(
    resized_gray2  /  255.0,
    orientations=9,
    pixels_per_cell=(8,  8),
    cells_per_block=(2,  2),
    block_norm="L2-Hys"
)
```

Step 21: Calculate HOG Distance

```
hog_distance = np.linalg.norm(
    resized_hog1 - resized_hog2
)

print(
    "\nHOG Distance after resizing:",
    round(hog_distance, 4)
)
```

```
HOG Distance after resizing: 28.3863
```

Step 22: Final Feature Summary

```
print("\n========== FEATURE EXTRACTION SUMMARY ==========")

print(
    "SIFT Keypoints - Image 1:",
    len(keypoints1)
)

print(
    "SIFT Keypoints - Image 2:",
    len(keypoints2)
)

print(
    "HOG Features - Image 1:",
    len(hog_features1)
)

print(
    "HOG Features - Image 2:",
    len(hog_features2)
)

if descriptors1 is not None and descriptors2 is not None:

    print(
        "Good SIFT Matches:",
        len(good_matches)
    )

print(
    "HOG Distance:",
    round(hog_distance, 4)
)
```

```
========== FEATURE EXTRACTION SUMMARY ==========
SIFT Keypoints - Image 1: 191
SIFT Keypoints - Image 2: 1227
HOG Features - Image 1: 1139544
HOG Features - Image 2: 1139544
Good SIFT Matches: 1
HOG Distance: 28.3863
```

Step 23: Observations

```
print("\n========== OBSERVATIONS ==========")

print(
```

```
    "1. SIFT detects distinctive keypoints in the images."
)

print(
    "2. SIFT descriptors represent local image features."
)

print(
    "3. SIFT is robust to changes in scale and rotation."
)

print(
    "4. HOG represents local gradient orientations."
)

print(
    "5. HOG is useful for describing object shape and edges."
)

print(
    "6. SIFT is suitable for feature matching."
)

print(
    "7. HOG is commonly useful for object detection."
)

print(
    "8. The number of detected features depends on image content."
)
```

```
========== OBSERVATIONS ==========
1. SIFT detects distinctive keypoints in the images.
2. SIFT descriptors represent local image features.
3. SIFT is robust to changes in scale and rotation.
4. HOG represents local gradient orientations.
5. HOG is useful for describing object shape and edges.
6. SIFT is suitable for feature matching.
7. HOG is commonly useful for object detection.
8. The number of detected features depends on image content.
```

Step 24: Final Output

```
print("\n========== EXPERIMENT 5 ==========")

print("Image Loading: Completed")
print("Grayscale Conversion: Completed")
print("SIFT Feature Extraction: Completed")
print("SIFT Keypoint Visualization: Completed")
print("HOG Feature Extraction: Completed")
print("HOG Visualization: Completed")
print("SIFT Image Matching: Completed")
print("Feature Comparison: Completed")
print("Observation Analysis: Completed")

print("\nExperiment Completed Successfully.")
```

```
========== EXPERIMENT 5 ==========
Image Loading: Completed
Grayscale Conversion: Completed
SIFT Feature Extraction: Completed
SIFT Keypoint Visualization: Completed
HOG Feature Extraction: Completed
HOG Visualization: Completed
SIFT Image Matching: Completed
Feature Comparison: Completed
Observation Analysis: Completed


Experiment Completed Successfully.
```

EXPERIMENT 5 - ANSWERS TO QUESTIONS

QUESTION 1: What is feature extraction, and why is it important in computer vision?

Feature extraction is the process of converting an image into meaningful numerical features or descriptors.

Instead of processing every pixel directly, feature extraction identifies important information such as:

Edges. Corners. Shapes. Textures. Local patterns.

Feature extraction is important because it provides a compact representation of an image that can be used for:

Object recognition. Image matching. Image classification. Object detection. Tracking.

Therefore, feature extraction is an important step in many computer vision systems.

QUESTION 2: Explain the working principle of Scale-Invariant Feature Transform (SIFT).

SIFT stands for Scale-Invariant Feature Transform.

SIFT detects distinctive features in an image and generates descriptors for those features.

The main steps are:

Scale-space construction: The image is examined at different scales.

Keypoint detection: Important points are detected using differences of Gaussian-blurred images.

Keypoint localization: Unstable points are removed and accurate keypoint locations are determined.

Orientation assignment: An orientation is assigned to each keypoint based on local image gradients.

Descriptor generation: A numerical descriptor is created around every keypoint.

These descriptors can then be compared between images for feature matching.

QUESTION 3: What are keypoints and feature descriptors in image analysis?

Keypoints are distinctive locations in an image.

Examples include:

Corners. Strong edges. Texture-rich points.

Keypoints are locations where useful visual information can be detected.

Feature descriptors are numerical representations of the image region around a keypoint.

In SIFT, each keypoint is represented by a descriptor that can be compared with descriptors from another image.

Therefore:

Keypoint → Important location. Descriptor → Numerical description of that location.

QUESTION 4: Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.

HOG stands for Histogram of Oriented Gradients.

HOG describes an image using the directions and strengths of local image gradients.

The main steps are:

Calculate image gradients. Divide the image into small cells. Calculate gradient orientation histograms. Group cells into blocks. Normalize the block histograms. Combine them into a feature vector.

HOG captures the shape and edge structure of objects.

It is especially useful for object detection because the shape of an object can often be represented using its gradient patterns.

QUESTION 5: Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

SIF T:

Detects local keypoints. Provides local feature descriptors. Robust to scale changes. Robust to rotation changes. Suitable for image matching. Generally more computationally expensive than simpler gradient descriptors.

HOG:

Represents gradient orientation patterns. Describes shape and edge information. Commonly used for object detection. Computationally simpler than SIFT in many applications. More dependent on image/object alignment.

Applications:

SIFT → Image matching, panorama creation, object recognition.

HOG → Object detection, pedestrian detection, shape analysis.

QUESTION 6:

Why is SIFT considered invariant to scale and rotation?

SIFT is considered scale invariant because it searches for keypoints across multiple image scales using a scale-space representation. Therefore, the same feature can be detected even if the object appears larger or smaller.

SIFT is considered rotation invariant because it assigns an orientation to each keypoint based on the local gradient directions.

The descriptor is then constructed relative to this assigned orientation.

Therefore, rotating an image generally does not prevent the same local feature from being matched.

QUESTION 7: Mention three real-world applications where HOG descriptors are commonly used.

Three applications of HOG are:

Pedestrian Detection: HOG can represent the shape and edge structure of pedestrians.

Vehicle Detection: HOG features can help identify characteristic shapes and edges of vehicles.

Object Detection: HOG descriptors can be used to represent object shapes for detection and classification.

HOG has historically been widely used in computer vision systems involving shape-based object detection.

QUESTION 8: Why is feature extraction performed before image classification or object detection?

Raw images contain a very large number of pixel values.

Feature extraction converts raw pixels into meaningful representations.

This can:

Reduce the complexity of image data. Highlight important visual information. Improve computational efficiency. Help classifiers identify useful patterns. Improve object detection performance.

Therefore, feature extraction can provide a more useful representation for classification and detection algorithms.

QUESTION 9: What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Advantages of Handcrafted Features:

Do not always require large training datasets. Usually have understandable mathematical properties. Can be computationally efficient for specific tasks. Can work well in controlled environments.

Limitations:

Require manually designed algorithms. May not generalize well to complex real-world conditions. Performance can depend strongly on parameter selection. May not capture high-level semantic information.

Deep Learning:

Deep learning models can automatically learn features from training data.

They can capture complex patterns and high-level semantic information.

However, deep learning often requires:

Large datasets. Significant computational resources. Longer training time.

Therefore, handcrafted features and deep learning each have advantages depending on the application and available data.

QUESTION 10: How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

Feature extraction provides useful numerical information from images.

Image Matching:

Local features such as SIFT keypoints and descriptors can be compared between two images to identify corresponding regions.

Face Recognition:

Feature descriptors can represent distinctive facial characteristics which can then be compared with stored representations.

Object Detection:

HOG and other feature descriptors can represent object shapes, edges and local patterns to help identify objects.

Therefore, feature extraction forms an important bridge between raw image data and computer vision algorithms.

